# IT457: CLOUD COMPUTING

**GROUP 36**

**TOPIC — DIGILOCKER**

**WEEK 2**





| Name | Student ID |
|---|---|
| Mahek Jikkar | 202301260 |
| Shambhavi Singh | 202301107 |
| Meet Jain | 202301073 |

# DigiLocker — Case Analysis and Proposed Architecture

## 1. Problem statement and case context

How do you run a national-scale document service in the cloud that stays available and fast during enormous, nationally synchronised demand spikes, when the documents themselves live in thousands of systems you do not control, under government data-residency rules, and with no revenue per user to pay for any of it?

DigiLocker is India's national digital document wallet, run by the National e-Governance Division under MeitY. A citizen signs in and reaches driving licences, vehicle registrations, PAN, marksheets and insurance policies that carry the same legal standing as the paper original. As of 31 July 2026 it had more than 72.43 crore registered users and offered 5,437 document types and services.

Four pressures shape the engineering.

**Load is event-driven, not steady.** Traffic is mostly flat, then a board result is declared and tens of millions of students arrive within the hour. CBSE published Class 12 results through DigiLocker in May 2026 and deliberately ran several result platforms at once to spread demand. The COVID vaccination certificate produced the same effect nationally.

**The platform is federated, not owned.** Documents are produced by central ministries, state transport authorities, examination boards and insurers, whose systems vary enormously in capacity. DigiLocker cannot upgrade any of them.

**Trust and sovereignty are hard constraints.** Documents must be verifiably authentic, access must be consented to and auditable, and data must remain on approved Indian infrastructure.

**There is no revenue per user.** Cost per transaction has to stay negligible, which explains more of the architecture than any other single factor.

### Scope of this report

This report contains two distinct architectures and keeps them separate throughout.

- **Architecture A** is how DigiLocker works today, reconstructed from the published specifications. It is descriptive. No official deployment diagram exists, so every inference is labelled.
- **Architecture B** is our proposed multi-region, high-availability design for a DigiLocker-like service. It is prescriptive, it is our own work, and it is not a claim about DigiLocker's real deployment.

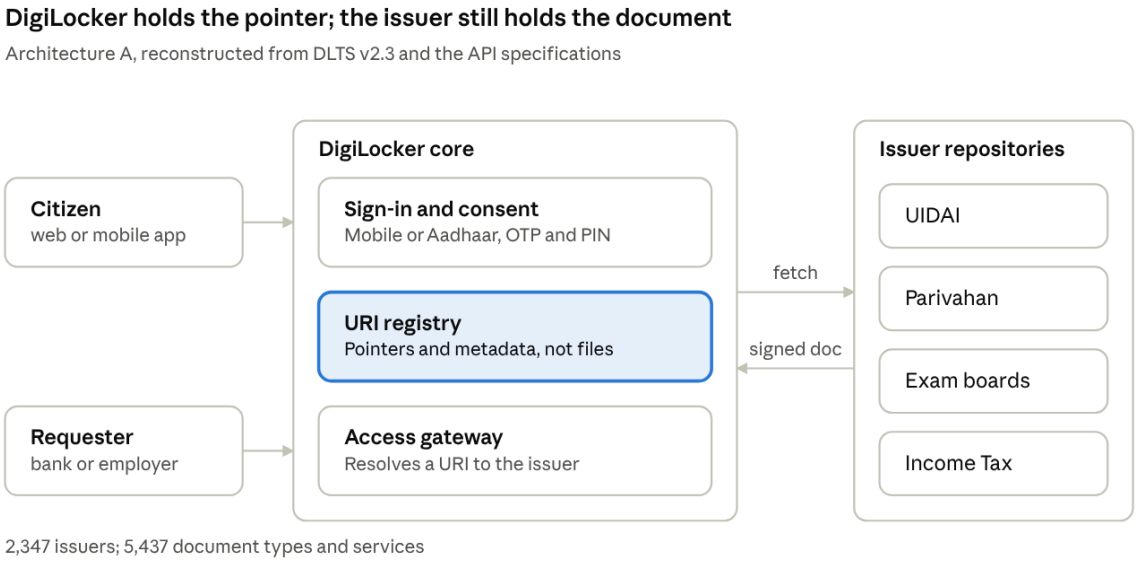
Sections 2 and 3 cover A and its failure modes. Sections 4 to 6 cover B, how we will evaluate it, and what we will build.

## 2. Architecture A: how DigiLocker works today

The Digital Locker Technology Specification v2.3 defines six roles: the issuer that produces a document, the digital repository that holds it, the digital locker belonging to the citizen, the requester that wants to verify it, the access gateway that connects them, and the document URI itself.

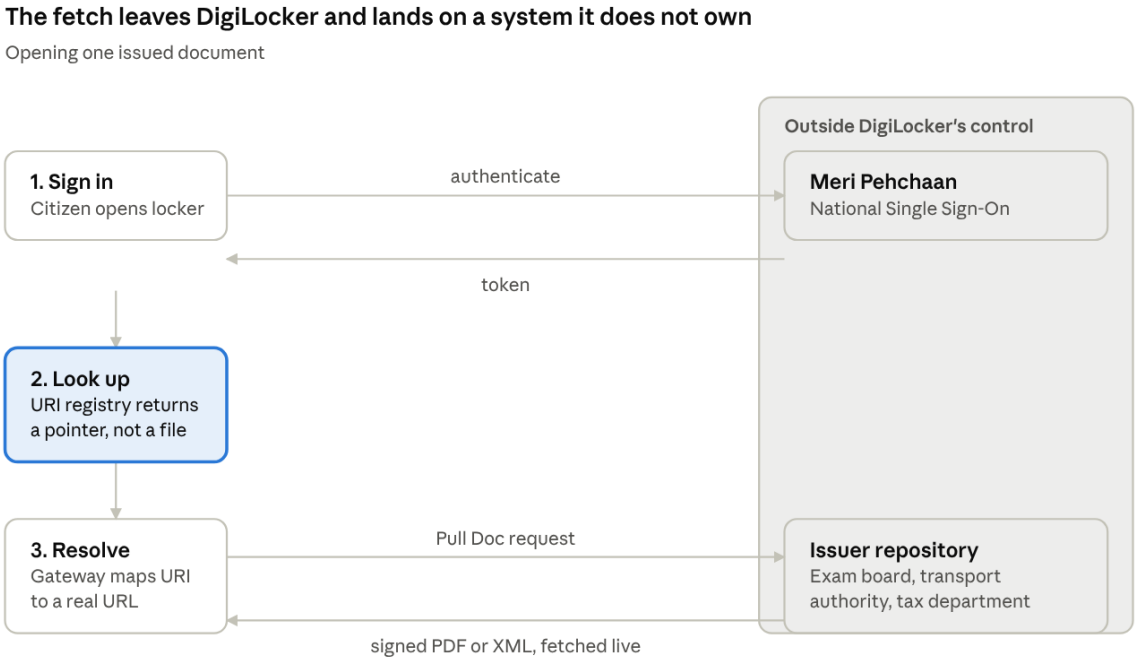
The stated design assumption is the one that matters. Documents are held in a federated way, with no

central document repository. The platform stores a pointer and resolves it when the request arrives.




Component view · 2 actors, 3 core services, federated issuers

## The request path




Issued-document fetch · 3 internal steps, 2 external systems

## Mechanics

- **The URI carries the design.** Every document has an identifier of the form `IssuerID-DocType-DocID`, independent of where the file sits. The gateway resolves it to a real URL in the correct repository

at request time.

- **Two issuance paths exist.** Most documents are pulled from the issuer on demand. Registered partner organisations can also push a certificate directly into a citizen's locker.
- **Self-uploaded documents are the exception.** DigiLocker stores these, capped at 1 GB per citizen, and they are explicitly not authentic originals. Only issued documents carry legal standing.
- **Consent precedes retrieval.** A requester sends the citizen to a DigiLocker login and consent step, then makes a separate call to retrieve the document. The two are distinct operations.
- **Documents are digitally signed XML** with a printable version attached, which is what makes a fetched copy verifiable rather than a scan.
- **The gateway is stateless about content.** DLTS requires it to store no document, no URI and no credentials, keeping only anonymised audit logs.
- **Loose coupling is mandated.** The specification requires repository, gateway and locker to communicate only through DLTS APIs, and gives the reason: each provider must be able to refactor, scale, administer and price independently. Independent scalability was a 2015 design decision, not a later retrofit.

Hosting is on government cloud. The first version ran at the National Informatics Centre, and the platform operates from ISO 27001 certified government data centres. NIC's GI Cloud, MeghRaj, serves national data centres at Delhi, Hyderabad, Pune and Bhubaneswar.

## 3. Failure analysis

Working from Architecture A, we enumerated the ways a platform of this shape degrades. Each failure mode below drives a specific decision in Section 4, and each is something our Week 3 build can actually reproduce.

| # | Failure mode | What happens | Why it matters here |
|---|---|---|---|
| F1 | Synchronised demand spike | Tens of millions of users arrive within minutes of a result declaration | Load is roughly 7 to 8 requests per second on average, so peak may be orders of magnitude above baseline |
| F2 | One slow issuer | A state department responds in seconds instead of milliseconds | Connections held open; thread pool exhausted; the whole platform degrades because of one dependency |
| F3 | One dead issuer | A repository returns errors or times out entirely | Without isolation, repeated retries amplify the load on an already failing system |
| F4 | Identity provider degradation | Meri Pehchaan is slow or unavailable | Nobody can sign in, so every downstream service is idle but the platform is effectively down |
| F5 | Availability zone loss | One zone becomes unreachable | Capacity halves at the worst possible moment if instances are not spread |
| F6 | Region loss | An entire region is unavailable | Requires cross-region failover, which must respect data-residency rules |
| F7 | Metadata store saturation | The pointer lookup becomes the bottleneck | Every request touches it, so it is the one component with no natural sharding by document |

### The one that defines this case

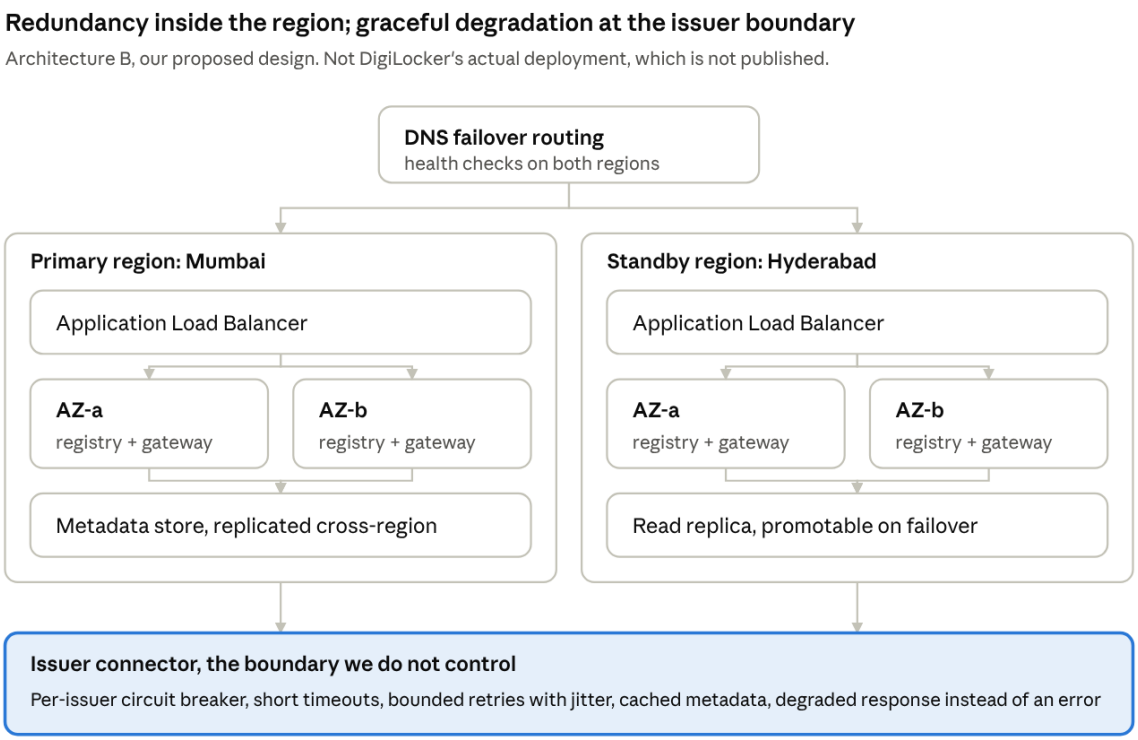
F2 is the interesting failure, and it is the reason we chose this exercise over a generic autoscaling demonstration.

Autoscaling solves F1. Adding instances handles a spike in arriving requests. But autoscaling makes F2 worse. If every instance holds connections open waiting on a slow issuer, adding instances adds more waiting connections against a dependency that is already struggling. The platform scales its way into a deeper failure.

This is why Section 4 treats the issuer boundary differently from everything else: timeouts, bounded retries with jitter, a circuit breaker per issuer, and graceful degradation rather than an error page. A design that only scales horizontally would pass a load test and still fail on result day.

## 4. Architecture B: proposed multi-region design

This is our design for a DigiLocker-like service, not a description of DigiLocker. It answers the failure modes in Section 3 directly.




Proposed multi-region design · 2 regions, 2 zones each, one shared issuer boundary

### Design decisions and their justification

**Two Indian regions, warm standby.** Mumbai as primary and Hyderabad as standby keeps all data inside India, which the sovereignty constraint requires. We chose warm standby from the four disaster-recovery strategies rather than multi-site active-active: active-active roughly doubles steady-state cost for a workload whose average load is modest, and backup-and-restore gives a recovery time measured in hours, which is unacceptable on a result day. Warm standby buys a recovery time in minutes at a

fraction of active-active's cost. **Answers F6.**

**Two availability zones per region, behind a load balancer.** Instances spread across zones so losing one removes half of capacity rather than all of it, with the autoscaling group replacing lost capacity automatically. **Answers F5.**

**Horizontal autoscaling on the stateless tier.** The registry and gateway hold no session state, so they scale out on request count. This is the straightforward part. **Answers F1.**

**A circuit breaker per issuer.** This is the central decision. Each issuer gets its own breaker, its own short timeout and its own bounded retry budget with jitter. When an issuer's error rate crosses a threshold, the breaker opens and requests to that issuer fail immediately rather than consuming connections. Crucially the breaker is per issuer, not global, so one failing state department cannot take down access to documents from the other 2,346. **Answers F2 and F3.**

**Graceful degradation rather than an error.** When a breaker is open, the citizen sees the document's metadata from cache with a clear note that the issuing department is temporarily unreachable, instead of a blank error. The document list still loads. This matters because a result-day outage is a political event, and partial service is substantially better than none.

**Cached document metadata, never cached documents.** Metadata is cached at the edge to keep the common path off the database. Documents themselves are never cached, because caching them would reintroduce exactly the storage cost and staleness risk the pointer design exists to avoid, and would weaken the authenticity guarantee.

**Identity treated as an external dependency.** Sign-in through a national SSO gets the same treatment as an issuer: timeout, breaker, and a cached session that survives a brief identity-provider outage so users already signed in are not ejected. **Answers F4.**

**Read replicas and connection pooling on the metadata store.** Every request touches it, so it gets read replicas in-region and a connection pool sized against instance count rather than left to default. **Answers F7.**

### What we are deliberately not doing

We are not proposing to centralise document storage. It would simplify availability but would discard the cost structure, the authenticity model and the federation that define the case. The constraint is the interesting part of the problem, not an obstacle to route around.

## 5. Cost model and evaluation metrics

### What is known about DigiLocker's cost

Nothing, in financial terms. No infrastructure cost, instance inventory or cost-per-transaction figure is published in any source we reviewed. Governments disclose budget allocations and adoption statistics, not cloud bills, and because hosting sits on government cloud the costs are internal allocations rather than a market bill.

What the sources do establish is the cost structure, which is more useful. The architecture removes the dominant line item. More than 941 crore documents have been made available through DigiLocker; storing, replicating and backing up that volume centrally would exceed every other cost combined. The pointer model means the platform never pays for it. What stays in the hot path, identity and metadata lookup, is small and stateless. That is why a platform with 72 crore users can run on modest public infrastructure.

### Our cost model

We will model cost from our own measured throughput using the AWS pricing calculator, covering compute, managed database, load balancing, data transfer and monitoring, and report it as cost per thousand requests. Two scenarios: steady state at measured baseline, and a result-day peak extrapolated from our load tests. This will be labelled a modelled estimate against Architecture B, not DigiLocker's spend.

We will also price the counterfactual: what object storage for a comparable document volume would cost. The gap between the two is the quantified value of the pointer design, and it is the most defensible number this project can produce.

### Metrics

| Metric | Why we measure it | Target or comparison |
|---|---|---|
| p50 and p95 latency | Averages hide the tail that users actually feel | p95 under 1 second at baseline |
| Latency against concurrency | Shows where the system bends | Identify the knee of the curve |
| Error rate | Distinguishes slow from broken | Under 1 percent at target load |
| Autoscaling reaction time | Time from threshold breach to healthy new instance | Measured, compared against spike arrival rate |
| Recovery time after zone loss | How long until full capacity returns | Measured |
| Behaviour with a degraded issuer | Does one slow dependency degrade everything | Breaker opens, other issuers unaffected |
| Cost per thousand requests | Makes the architecture comparable | Modelled from measured throughput |

The sixth row is the one that distinguishes this project. The others are standard load-testing output.

## 6. Exercise plan

The brief for government and public platform cases asks us to design a multi-region, high-availability architecture and then build a scaled-down version of one component. Section 4 is the design. This section is the build.

### Scope

Multi-region is a paper deliverable. What we build is multi-zone within one AWS region, which is what the free tier supports and what the brief asks for. We will say so explicitly in the results rather than imply we tested cross-region failover.

### What gets built

A miniature of the DigiLocker core, in Python with FastAPI. Deliberately small, because the infrastructure is the subject and the application is the instrument.

- **Registry service.** Authenticates a request, looks up a document URI in a managed database, calls the issuer, returns the document. Holds pointers, never files.
- **Mock issuer service.** A separate deployment, on its own instance, that returns a signed-XML-shaped payload. Exposes controls to inject latency, error rates and total failure. This is the instrument for F2 and F3.
- **A second mock issuer**, healthy throughout, to prove that breaker isolation is per issuer rather than global.

### AWS components

Application Load Balancer; an autoscaling group of `t3.micro` instances across two availability zones; a managed database for the pointer table; the mock issuers on separate instances; CloudWatch for metrics. Load generated with k6 from outside the VPC.

### Scenarios

Stating the expected result beforehand is what makes this an experiment rather than a demonstration. We record whether each expectation held.

| # | Scenario | We measure | We expect |
|---|---|---|---|
| E1 | Baseline, steady low load | p50, p95, error rate, cost per thousand requests | Flat latency, no scaling events, our control case |
| E2 | Ramp to peak concurrency | Latency against concurrency, autoscaling reaction time | Latency flat, then a knee; new instances healthy in 2 to 4 minutes, slower than the arrival rate |
| E3 | Terminate one instance mid-test | Error rate during the gap, time to full capacity | Brief error spike, load balancer drains the unhealthy target, ASG replaces it |
| E4 | Issuer injected with 3 second latency, then failure | Latency on the affected issuer, latency on the healthy issuer, breaker state | Without a breaker: both degrade. With one: affected issuer fails fast, healthy issuer unaffected |

E4 is run twice, once with the circuit breaker disabled and once enabled. That contrast is the result the whole project is built to produce, and it is the one slide worth showing in the VIVA.

### What would make this fail

If E4 with the breaker enabled shows no meaningful difference, either our load is too low to exhaust connections or the timeout is doing the breaker's work. We would report that rather than adjust the experiment until it agrees with Section 4.

## 7. Repository, deployment and live hosting

### Repository

| | |
|---|---|
| `/app` | registry service |
| `/mock-issuer` | controllable issuer simulator |
| `/infra` | deployment scripts, security group and ASG configuration |
| `/loadtest` | k6 scripts, one per scenario |
| `/results` | raw CloudWatch exports and k6 output, committed as produced |
| `/docs` | week 1 summary, case analysis, this report, research log |
| `README.md` | what it is, the live URL, how to run it |

## 8. Roles and schedule

Roles are defined by function.

| Role | Owns | Deliverable |
|---|---|---|
| Cloud lead | The AWS account, all deployment, the billing alarm, and the standing live URL | Working infrastructure and the live service |
| Application developer | Registry service and both mock issuers, including the fault-injection controls | Code in the repository, reviewed |
| Load testing and metrics | k6 scripts for E1 to E4, CloudWatch collection, raw results | Results committed as produced |
| Research and architecture | The specifications, Architecture A, the failure analysis | Sections 2 and 3, and the reconstruction's accuracy |
| Cost and report | Cost model, pricing calculator work, report assembly and consistency | Final report, cost section |

## 9. References

## Primary sources on the case

| Source | Publisher, date | Link |
|---|---|---|
| Digital Locker Technology Specification v2.3 | NeGD, Government of India, Mar 2015 | PDF |
| DigiLocker Issuer API Specification v1.13 | NeGD, MeitY, May 2024 | PDF |
| DigiLocker Requester API Specification v2.1 | NeGD, MeitY, Oct 2016 | PDF |
| DigiLocker Authorized Partner API Specification v1.13 | NeGD, MeitY, Feb 2021 | PDF |
| DigiLocker Partners Resource Centre | API Setu, NeGD | Page |
| PIB: UMANG and DigiLocker service delivery | Press Information Bureau, 12 Aug 2026 | Release |
| MeitY Annual Report 2025-26 | MeitY, Apr 2026 | PDF |
| PIB: cloud infrastructure for digital governance | Press Information Bureau, 2025 | Release |
| DigiLocker: Citizen's Vault of Trust on Public Cloud | NeGD, MeitY, Oct 2025 | PDF |
| DigiLocker: The Digital Briefcase for India's Authentic E-Documents | NeGD, MeitY, 2025 | Article |

## Cloud engineering references for Architecture B

These informed the proposed design in Section 4. Each team member should read the sections relevant to their role rather than the whole document.

- AWS Well-Architected Framework, Reliability Pillar. Used for the workload architecture and failure management principles.
- AWS disaster recovery whitepaper. Source of the four strategies and their recovery-time trade-offs; our warm standby choice in Section 4 is argued against these.
- Amazon Builders' Library, "Timeouts, retries and backoff with jitter". Directly informs the issuer connector design and the retry budget.
- Microsoft Azure Architecture Center, Circuit Breaker pattern. The reference description of the per-issuer breaker behaviour in Section 4.

These are engineering references for our own design. None of them is evidence about how DigiLocker is actually built.